

Set 7 - Variables, Operators and Type Casting

Q1.

Declare variables in Java to represent a Flipkart product: productName (String), price (double), inStock (boolean), and rating (float). Initialize each with sample values and print them.

Answer:

```
public class FlipkartProductVariables {
    public static void main(String[] args) {
        String productName = "Noise Cancelling Headphones";
        double price = 2999.99;
        boolean inStock = true;
        float rating = 4.5f;

        System.out.println("Product Name: " + productName);
        System.out.println("Price: Rs. " + price);
        System.out.println("In Stock: " + inStock);
        System.out.println("Rating: " + rating + " / 5");
    }
}
```

```
/* Sample Output:
Product Name: Noise Cancelling Headphones
Price: Rs. 2999.99
In Stock: true
Rating: 4.5 / 5
*/
```

Q2.

Write a Java program that takes a user's Zomato wallet balance (double) and applies a 10% cashback if the balance is above Rs.500, otherwise no cashback. Use arithmetic and relational operators to calculate and display the final balance.

Answer:

```
public class ZomatoCashback {
    public static void main(String[] args) {
        double balance = 750.0;
        double finalBalance;

        if (balance > 500) {
            double cashback = balance * 0.10;    // arithmetic operator
            finalBalance = balance + cashback;
            System.out.println("Cashback of Rs. " + cashback + " applied!");
        } else {
            finalBalance = balance;
            System.out.println("No cashback applied.");
        }

        System.out.println("Final wallet balance: Rs. " + finalBalance);
    }
}
```

```
/* Sample Output:
Cashback of Rs. 75.0 applied!
```

```
Final wallet balance: Rs. 825.0
*/
```

Q3.

Create a Java snippet that checks if a user can book a movie ticket on BookMyShow: declare variables for userAge (int) and hasPaymentMethod (boolean). Use logical and relational operators to print 'Booking allowed' only if the user is 18 or older and has a payment method linked.

Answer:

```
public class BookMyShowEligibility {
    public static void main(String[] args) {
        int userAge = 20;
        boolean hasPaymentMethod = true;

        if (userAge >= 18 && hasPaymentMethod) {
            System.out.println("Booking allowed");
        } else {
            System.out.println("Booking not allowed");
        }
    }
}

/* Sample Output:
Booking allowed
*/
```

Q4.

Write a Java program that takes an integer representing the number of unread WhatsApp messages and prints 'No new messages', 'Few messages', or 'Too many messages' using the ternary operator. Hint: Use nested ternary operators for three possible outputs.

Answer:

```
public class WhatsAppMessages {
    public static void main(String[] args) {
        int unreadMessages = 45;

        String status = (unreadMessages == 0) ? "No new messages"
            : (unreadMessages <= 20) ? "Few messages"
            : "Too many messages";

        System.out.println(status);
    }
}

/* Sample Output (for unreadMessages = 45):
Too many messages
*/
```

Q5.

Demonstrate both implicit and explicit type casting in Java by converting a float rating (e.g., 4.7) to an int and then back to double. Print all values to show the effect of casting. Constraint: Do not use any external libraries; use only standard Java casting syntax.

Answer:

```
public class TypeCastingDemo {
    public static void main(String[] args) {
        float rating = 4.7f;
        System.out.println("Original float rating: " + rating);

        // Explicit casting: float -> int (narrowing, decimal part is truncated)
        int intRating = (int) rating;
        System.out.println("After explicit cast to int: " + intRating);

        // Implicit casting: int -> double (widening, done automatically by Java)
        double doubleRating = intRating;
        System.out.println("After implicit cast to double: " + doubleRating);
    }
}
```

/* Sample Output:

Original float rating: 4.7

After explicit cast to int: 4

After implicit cast to double: 4.0

*/

// Note: Casting float to int truncates (does not round) the decimal part,
// which is why 4.7 becomes 4, not 5.